

Sezione 1 - Programmazione in RUST

Progetto 2.1: Ruggine: App di Chat Testuale

Realizzare un'applicazione, chiamata Ruggine, di tipo client/server per gestire una chat di scambio di messaggi testuali. La chat deve prevedere la possibilità di creare gruppi di utenti per la condivisione di messaggi.

L'ammissione alla Chat è effettuata al primo avvio del programma, inviando al server una richiesta di iscrizione, mentre l'ingresso in un gruppo avviene solo su invito.

Il programma deve girare su almeno 2 tra le diverse piattaforme disponibili (Windows, Linux, MacOS, Android, ChromeOS, iOS).

Si richiede di porre attenzione alle prestazioni del sistema in termini di consumo di tempo di CPU e di dimensione dell'applicativo.

L'applicazione deve generare un file di log, riportando, ogni 2 minuti, i dettagli sul tempo di CPU utilizzato da parte del server.

Si richiede inoltre di riportare nel report descrittivo del progetto la dimensione del file eseguibile.

Progetto 2.2: **Hardened Code**: Preprocessing per l'irrobustimento del codice e Sviluppo di Ambiente di Fault Injection

Si chiede di realizzare un ambiente di fault injection per la valutazione della tolleranza ai guasti di un'applicazione che è stata irrobustita con la duplicazione automatica delle variabili e/o di funzioni.

Il progetto mira a raggiungere 2 obiettivi:

1. Modificare il codice sorgente attraverso la duplicazione di variabili e/o di porzioni di codice ed il controllo della congruenza del contenuto delle variabili dopo ogni operazione di lettura e del valore di ritorno delle funzioni ridondate attraverso l'esecuzione di un applicativo di pre-processing come spiegato successivamente
2. Realizzare un ambiente di Fault Injection costituito da:
 - a. Un programma ridonato come descritto più dettagliatamente nel seguito
 - b. Una Lista di guasti caratterizzati dal nome della variabile, dal tempo di iniezione e dal bit modificato
 - c. Un Iniettore che scandisce la lista di guasti e per ogni guasto lancia il programma in cui esso verrà iniettato
 - d. Un Analizzatore che raccoglie i dati sulla base del risultato dell'elaborazione.

L'applicativo di preprocessing deve leggere il codice sorgente originale (scritto in RUST) e deve riconoscere la presenza dell'attributo `#[DUPLICATE]` che precede la dichiarazione della variabile o della funzione da duplicare.

L'applicativo di preprocessing dovrà modificare il codice sorgente gestendo i seguenti casi:

1. le variabili con l'attributo indicato devono essere duplicate. Si ricorda che tali variabili duplicate determinano la duplicazione di tutti i riferimenti ad esse e l'eventuale duplicazione di tutti i movimenti. Per verificare la correttezza del codice in esecuzione ogni accesso alla variabile in lettura richiede il controllo delle 2 copie e in caso di errore il programma deve generare una condizione di panic, con il dettaglio della variabile che presenta un errore
2. Le funzioni con l'attributo indicato devono essere eseguite 2 volte su argomenti duplicati. Le 2 esecuzioni forniranno 2 valori di ritorno che andranno controllati ed in caso di errore il programma deve generare una condizione di panic con il dettaglio della funzione che presenta un errore.

Ad ogni gruppo partecipante verranno distribuiti qualche programma di esempio da utilizzare come benchmark.

L'applicazione deve girare su un'unica piattaforma a propria scelta.